

Compile, link and run the program with `gcc -Wall -o callback callback.c reg_callback.c` and `./callback`:

```
This is a program demonstrating function callback
inside register_callback
inside my_callback
back inside main program
```

The code needs a little explanation. Assume that we have to call a callback function that does some useful work (error handling, last-minute clean-up before exiting, etc), after an event occurs in another part of the program. The first step is to register the callback function, which is just passing a function pointer as an argument to some other function (e.g., `register_callback`) where the callback function needs to be called.

We could have written the above code in a single file, but have put the definition of the callback function in a separate file to simulate real-life cases, where the callback function is in the top layer and the function that will invoke it is in a different file layer. So the program flow is like what can be seen in Figure 1.

The higher layer function calls a lower layer function as a normal call and the callback mechanism allows the lower layer function to call the higher layer function through a pointer to a callback function.

This is exactly what the Wikipedia definition states.

Use of callback functions

One use of callback mechanisms can be seen here:

```
/* This code catches the alarm signal generated from the kernel
   Asynchronously */
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

struct sigaction act;

/* signal handler definition goes here */
void sig_handler(int signo, siginfo_t *si, void *ucontext)
{
    printf("Got alarm signal %d\n", signo);
    /* do the required stuff here */
}

int main(void)
{
    act.sa_sigaction = sig_handler;
    act.sa_flags = SA_SIGINFO;

    /* register signal handler */
    sigaction(SIGALRM, &act, NULL);
    /* set the alarm for 10 sec */
```

```
alarm(10);
/* wait for any signal from kernel */
pause();
/* after signal handler execution */
printf("back to main\n");
return 0;
}
```

Signals are types of interrupts that are generated from the kernel, and are very useful for handling asynchronous events. A signal-handling function is registered with the kernel, and can be invoked asynchronously from the rest of the program when the signal is delivered to the user process. Figure 2 represents this flow.

Callback functions can also be used to create a library that will be called from an upper-layer program, and in turn, the library will call user-defined code on the occurrence of some event. The accompanying source code (`insertion_main.c`, `insertion_sort.c` and `insertion_sort.h`), which you can download from http://linuxforu.com/article_source_code/feb12/source_code.zip shows this mechanism used to implement a trivial insertion sort library. The flexibility lets users call any comparison function they want. Do download and play around with the same, since it is omitted from this article due to space constraints.

Wrapping it up

The tutorial describes how callbacks are implemented in C. In C++ the same goal can be achieved through template functors. Although callbacks are very hard to trace in large real-world programs, life should be a little easier after going through this introductory article. Just remember, callback is nothing but passing the function pointer to the code from where you want your handler/callback function to be invoked. Happy hacking! 

Reference

- [1] *Introduction to Algorithms*, 3/e by Thomas H Cormen, Charles E Leiserson, Ronald L Rivest and Clifford Stein
- [2] [http://en.wikipedia.org/wiki/Callback_\(computer_programming\)](http://en.wikipedia.org/wiki/Callback_(computer_programming))
- [3] *Linux System Programming* by Robert Love
- [4] <http://www.learnccp.com/cpp-tutorial/78-function-pointers/>

By: Dibyendu Roy

The author is a Linux fundamentalist. He works as a Linux developer with a large organisation, and spends most of his time studying and exploring the uncharted spaces of C and Linux. He dreams of becoming a guitar player some day, and spends precious moments with *LP, Cradle Of Filth* and *Children Of Bodom*. Being an open source activist, he uses Linux for every silly little thing. He can be contacted at diby.roy@gmail.com.